Registro de decisiones de arquitectura (ADR)

Proyecto: AutoFix Pro

Materia: Componentes de Lógica de Negocio · FIME UANL · **Profesor:** Miguel Ángel Gómez Marroquín

Equipo 6: Jesus Eduardo Moreno Osoria (2136063) · Osvaldo Salvador Cárdenas Mejorado (2141553) · Abraham Sandoval De León (2136604)

Este documento reúne las cinco decisiones de arquitectura del diseño de AutoFix Pro. Cada ADR sigue los campos que pide la guía (título, estado, contexto, alternativas, decisión, consecuencias y trazabilidad) y cita la evidencia de la Unidad 2 que la respalda. El informe de arquitectura presenta un resumen; aquí está el texto completo de cada decisión.

Resumen

ADR	Decisión	Estado	Necesidades
ADR-001	Domain Model como patrón de lógica de negocio	Aceptada	N-02 a N-04, N-07, N-10, N-11
ADR-002	Límites de los Aggregates y Unidad de Trabajo para CU-03	Aceptada	N-03, N-06, N-10
ADR-003	Service Layer por caso de uso y CQRS básico	Aceptada	N-02 a N-05, N-08, N-10, N-13
ADR-004	Repository en términos del dominio; persistencia en memoria en esta fase y base de datos del Data Center en el proyecto final	Aceptada	N-01 a N-03, CE-01, CE-06
ADR-005	La regla de $15,000 MXN como política del dominio	Aceptada	N-04, US-005, CA-007

ADR-001 — Domain Model como patrón de lógica de negocio

Campo	Contenido
Título	Usar Domain Model para el lado de escritura del proceso principal
Estado	Aceptada
Contexto	El proceso cotización → autorización → refacciones → entrega tiene 12 estados de orden y 6 de cotización (Glosario, «Estados del sistema»), nueve restricciones de proceso (BRD) y criterios de rechazo explícitos (CA-007, CA-009, CA-012, CA-020, CA-029, CA-042). Varias reglas dependen del estado y cruzan conceptos (cotización, orden, cliente). Es un proyecto universitario con alcance acotado.
Alternativas	**Transaction Script:** el más rápido, pero repetiría las validaciones de estado en cada caso de uso y no impediría que una función se salte una regla. **Table Module:** agrupa por tabla, pero las reglas que cruzan orden, cotización y cliente no tienen un lugar natural. **Domain Model:** cada regla vive en el objeto que protege sus datos; requiere más clases.
Decisión	Domain Model para los Aggregates Orden de Servicio, Cotización y Cliente. Las consultas de lectura (CQRS Queries) se resuelven con un lector simple, al estilo Transaction Script, porque solo arman vistas.
Consecuencias	(+) Las reglas se escriben una sola vez y se prueban sin base de datos (20 pruebas). (+) Los nombres del código son los del Glosario. (−) Más clases y mapeo entre los datos de entrada y el dominio. (−) CU-03 modifica dos Aggregates y necesita Unidad de Trabajo (ADR-002). Trabajo posterior: extender el modelo a recepción, diagnóstico y caja.
Trazabilidad	N-02, N-03, N-04, N-07, N-10, N-11 · US-004, US-005, US-006, US-010, US-016, US-024 · CA-005, CA-007, CA-010, CA-020, CA-029, CA-042 · BPMN Fases 2 a 6

ADR-002 — Límites de los Aggregates y Unidad de Trabajo para CU-03

Campo	Contenido
Título	Orden de Servicio y Cotización como Aggregates separados; la Solicitud de Compra dentro de la Orden
Estado	Aceptada
Contexto	Una orden puede tener varias cotizaciones (inicial v.1 y adicionales v.2+; US-009). El Gerente revisa cotizaciones sin tocar la orden (US-005). La decisión del cliente cambia la cotización y crea los trabajos de la orden (CA-010). La orden debe estar «En Espera de Refacción» mientras exista una solicitud «Solicitada» (CA-019, CA-022).
Alternativas	**A) Un solo Aggregate Orden con sus cotizaciones dentro:** consistencia total, pero un objeto enorme y el Gerente tendría que cargar toda la orden. **B) Cotización separada y Solicitud de Compra como Aggregate propio:** Compras independiente, pero el invariante de «En Espera de Refacción» quedaría entre dos Aggregates. **C) Cotización separada y Solicitud dentro de la Orden (elegida).**
Decisión	Opción C. La relación entre Aggregates es solo por identidad (numero_orden, id_cliente). Cuando un caso de uso modifica dos Aggregates (CU-03, y CU-01 al crear la cotización y marcar la orden), ambos cambios se confirman en la misma Unidad de Trabajo.
Consecuencias	(+) Cada Aggregate es pequeño y protege sus invariantes. (+) La regla de la espera de refacciones se cumple en una sola transacción. (−) CU-03 rompe la recomendación de «un Aggregate por transacción»; lo aceptamos porque el BRD exige evidencia consistente de la autorización (CE-01) y la Unidad de Trabajo lo garantiza (prueba test_ca029_en_flotilla_el_conductor_no_puede_autorizar). (−) Compras consulta las solicitudes a través del lector, no de un repositorio propio.
Trazabilidad	N-03, N-06, N-10 · US-006, US-009, US-010, US-011 · CA-010, CA-019, CA-022 · BPMN Fases 3 y 4

ADR-003 — Service Layer por caso de uso y CQRS básico

Campo	Contenido
Título	Un Application Service por caso de uso; Commands y Queries separados sin bases de datos distintas
Estado	Aceptada
Contexto	Los cinco casos de uso tienen actores distintos (Asesor, Gerente, Técnico). Los paneles de Compras, del Gerente y la consulta del cliente (US-011, US-005, US-021) solo leen y cruzan varias órdenes. La guía pide que la Service Layer coordine sin absorber reglas.
Alternativas	**Un único ServicioDeOrdenes:** menos archivos, pero mezcla actores y crece sin control. **Reglas en los servicios (Transaction Script disfrazado):** contradice ADR-001. **CQRS completo con base de lectura separada:** excesivo para el volumen de un taller.
Decisión	Cinco servicios (ServicioDeCotizacion, ServicioDeAprobacionGerencial, ServicioDeAutorizacion, ServicioDeRefacciones, ServicioDeEntrega), cada uno con un método que recibe un Command. Las consultas usan Queries con su manejador y la interfaz LectorDeOrdenes, y devuelven vistas (DTO) propias. Mismo almacenamiento para lectura y escritura.
Consecuencias	(+) Cada servicio tiene una sola razón para cambiar (SOLID-S). (+) Las vistas no exponen los Aggregates; la consulta del cliente no revela datos si el folio y las placas no coinciden (CA-038). (−) Más clases pequeñas. Trabajo posterior: agregar las consultas del Gerente (US-017, US-018) al mismo lector.
Trazabilidad	N-02 a N-05, N-08, N-10, N-13 · US-005, US-011, US-017, US-018, US-021 · CA-007, CA-021, CA-037, CA-038

ADR-004 — Repository en términos del dominio; persistencia en memoria en esta fase y base de datos del Data Center en el proyecto final

Campo	Contenido
Título	Interfaces de Repository en el dominio; implementación en memoria para validar el diseño; la base de datos del Data Center en la implementación
Estado	Aceptada
Contexto	El caso es ficticio y la guía indica que en esta unidad no es necesario construir bases de datos. Para el proyecto final, el profesor proporcionará un Data Center; el gestor de base de datos todavía no está definido. El BRD exige guardar digitalmente las autorizaciones (CE-01) y los documentos generados.
Alternativas	**Elegir ya un gestor y acceder a él desde los servicios:** amarra el diseño a una tecnología que aún no conocemos. **Usar un ORM dentro del dominio:** mezcla las reglas del negocio con detalles del framework. **Repository + Unidad de Trabajo como interfaces, con implementación en memoria por ahora (elegida).**
Decisión	Los repositorios (RepositorioDeOrdenes, RepositorioDeCotizaciones, RepositorioDeClientes) son interfaces del dominio con métodos de negocio (obtener, agregar, guardar, siguiente_numero). En esta fase se implementan en memoria. En el proyecto final se crearán implementaciones para la base de datos del Data Center, sin cambiar el dominio ni la aplicación.
Consecuencias	(+) El diseño no depende de un gestor que todavía no conocemos. (+) Cuando el Data Center esté disponible, solo se agrega código en la capa de infraestructura. (−) En esta fase los datos no se conservan al cerrar el programa; es aceptable porque solo se valida el diseño. Trabajo posterior: implementar los repositorios del Data Center cumpliendo el mismo contrato (por ejemplo, obtener() responde con OrdenNoEncontrada si no existe).
Trazabilidad	N-01 a N-03 · CE-01, CE-06 · BRD, Restricciones técnicas (almacenar digitalmente documentos)

ADR-005 — La regla de $15,000 MXN como política del dominio

Campo	Contenido
Título	Modelar la aprobación gerencial como PoliticaDeAprobacionGerencial inyectable
Estado	Aceptada
Contexto	El BRD establece que la cotización inicial mayor a $15,000 MXN requiere aprobación del Gerente antes de presentarse, y en la Unidad 2 se decidió que no aplica a las cotizaciones adicionales. Es la regla con más probabilidad de cambiar (monto o alcance).
Alternativas	**Condicional en ServicioDeCotizacion:** la aplicación decidiría el negocio. **Constante dentro de Cotizacion:** funciona, pero cada política nueva edita la clase. **Interfaz de política inyectada en Cotizacion.generar (elegida).**
Decisión	PoliticaDeAprobacionGerencial (interfaz) con la implementación PoliticaPorMontoDeCotizacionInicial(umbral = $15,000). Cotizacion.generar la consulta y decide el estado inicial.
Consecuencias	(+) Cumple Open/Closed: otra política es una clase nueva. (+) La regla se prueba de forma aislada (test_ca007). (−) Una abstracción más que explicar.
Trazabilidad	N-04 · US-005 · CA-007, CA-016 · BRD, Restricciones de proceso · BPMN gateway «¿Monto mayor a $15,000 MXN?»